Data Structures and Algorithms
Lecture 5 – Linked List-Part 2

M.G.Noel A.S Fernando (PhD)

Professor

ucsc

Deleting a Node from a Linked List

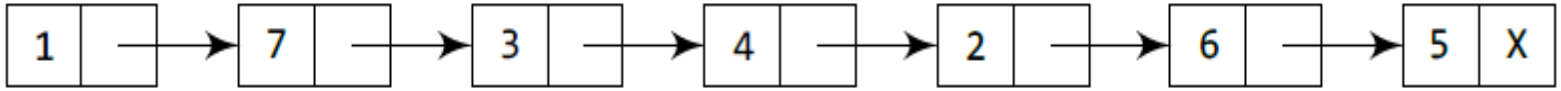
- When one deleting a node from a linked list, there are three cases are existing.
- Case 1: The first node is deleted.
- Case 2: The last node is deleted.
- Case 3: The node after a given node is deleted.

Deleting a Node from a Linked List- Case 1 : The first node is deleted

- Boundary condition: underflow occurs
- Underflow is a condition that occurs when we try to delete a node from a linked list that is empty.
- This happens when START = NULL or when there are no more nodes to delete.
- Note that when we delete a node from a linked list, we have to free the memory occupied by that node. The memory is returned to the free pool so that it can be used to store other programs and data.
- Whatever be the case of deletion, we always change the AVAIL pointer so that it points to the address that has been recently vacated.

Case 1: ***Deleting the First Node from a Linked List***

Original Linked List

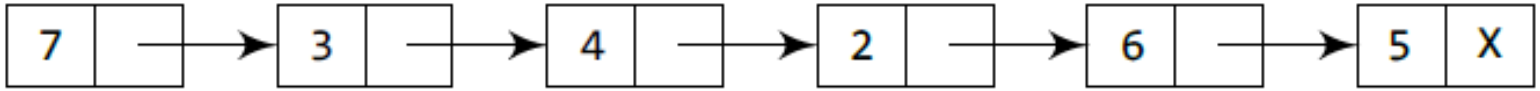


START

Make START to point to the next node in sequence.

After Deleting the First Node

Make START to point to the next node in sequence.



START

Cased 1: ***Deleting the First Node from a Linked List cont….***

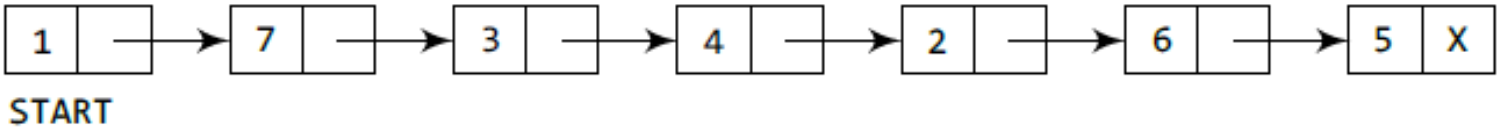
```
Step 1: IF START = NULL
            Write UNDERFLOW
            Go to Step 5
       [END OF IF]
Step 2: SET PTR = START
Step 3: SET START = START -> NEXT
Step 4: FREE PTR
Step 5: EXIT
```

Case 1: ***Deleting the First Node from a Linked List cont….***

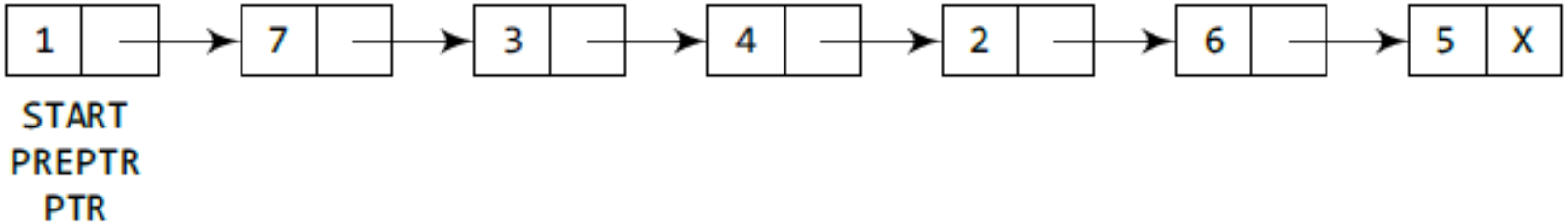
- if the linked list exists or not. If START = NULL, then it signifies that there are no nodes in the list and the control is transferred to the last statement of the algorithm.
- if there are nodes in the linked list, then we use a pointer variable PTR that is set to point to the first node of the list.
- For this, we initialize PTR with START that stores the address of the first node of the list.
- In Step 3, START is made to point to the next node in sequence and finally the memory occupied by the node pointed by PTR (initially the first node of the list) is freed and returned to the free pool.

Case 2: Deleting the Last Node from a Linked List

- Suppose we want to delete the last node from the linked list:

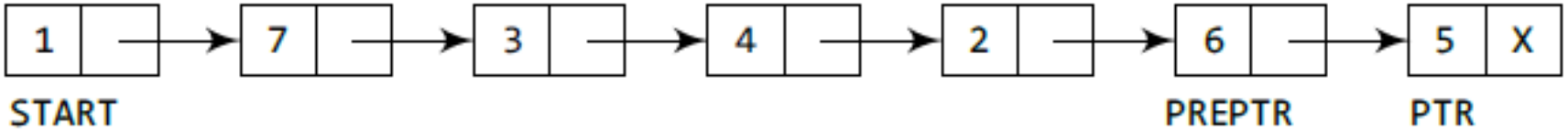


Take pointer variables PTR and PREPTR which initially point to START.

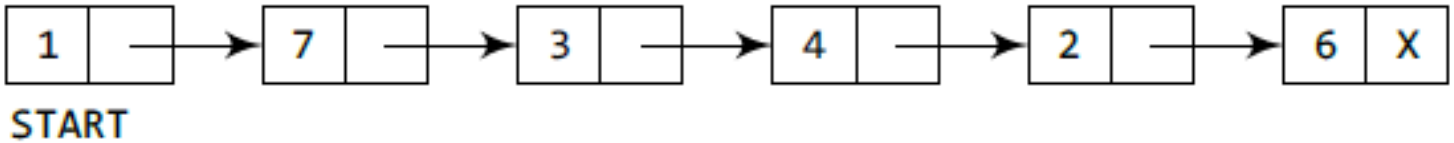
- 

Case 2: Deleting the Last Node from a Linked List cont.....

Move PTR and PREPTR such that NEXT part of PTR = NULL. PREPTR always points to the node just before the node pointed by PTR.



Set the NEXT part of PREPTR node to NULL.



Case 2: Deleting the Last Node from a Linked List cont…..

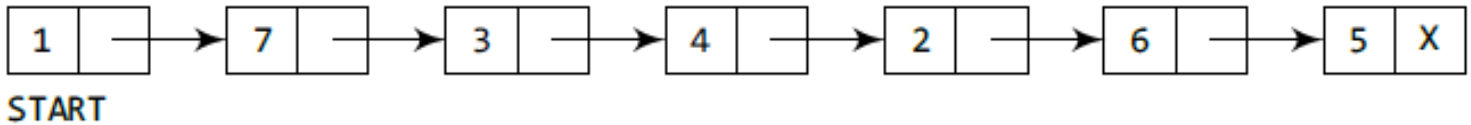
- we take a pointer variable PTR and initialize it with START. That is, PTR now points to the first node of the linked list.
- In the while loop, we take another pointer variable PREPTR such that it always points to one node before the PTR.
- Once we reach the last node and the second last node, we set the NEXT pointer of the second last node to NULL, so that it now becomes the (new) last node of the linked list.
- The memory of the previous last node is freed and returned back to the free pool.

Case 2: Deleting the Last Node from a Linked List cont.....

```
Step 1: IF START = NULL
            Write UNDERFLOW
            Go to Step 8
       [END OF IF]
Step 2: SET PTR = START
Step 3: Repeat Steps 4 and 5 while PTR -> NEXT != NULL
Step 4:     SET PREPTR = PTR
Step 5:     SET PTR = PTR -> NEXT
       [END OF LOOP]
Step 6: SET PREPTR -> NEXT = NULL
Step 7: FREE PTR
Step 8: EXIT
```

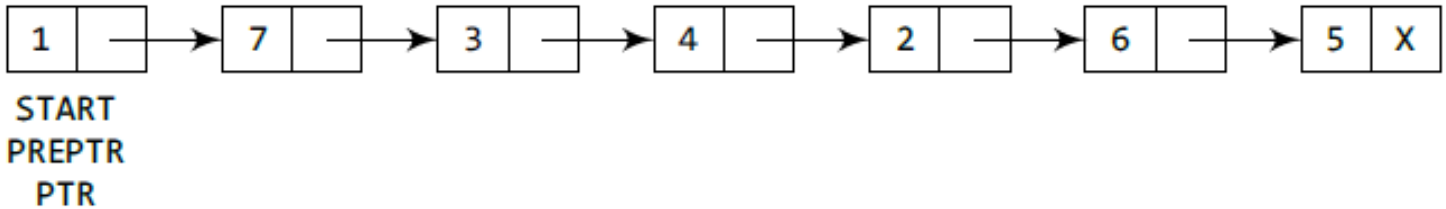
Case 3: ***Deleting the Node After a Given Node in a Linked List***

- For example, Suppose we want to delete the node that succeeds the node which contains data value 4 (i.e., node to be deleted is 2).



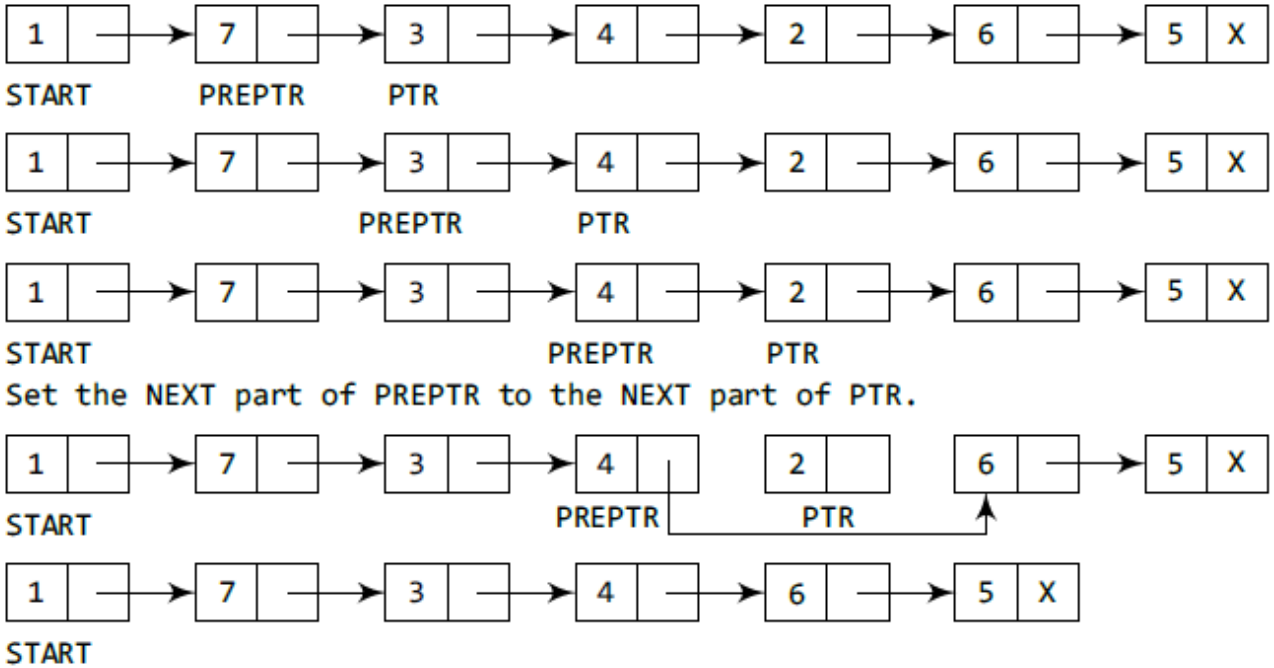
- Then the following changes will be done in the linked list.

Take pointer variables PTR and PREPTR which initially point to START.



Case 3: ***Deleting the Node After a Given Node in a Linked List cont....***

Move PREPTR and PTR such that PREPTR points to the node containing VAL and PTR points to the succeeding node.



Case 3: ***Deleting the Node After a Given Node in a Linked List cont....***

```
Step 1: IF START = NULL
            Write UNDERFLOW
            Go to Step 10
       [END OF IF]
Step 2: SET PTR = START
Step 3: SET PREPTR = PTR
Step 4: Repeat Steps 5 and 6 while PREPTR->DATA != NUM
Step 5:     SET PREPTR = PTR
Step 6:     SET PTR = PTR->NEXT
       [END OF LOOP]
Step 7: SET TEMP = PTR
Step 8: SET PREPTR->NEXT = PTR->NEXT
Step 9: FREE TEMP
Step 10: EXIT
```

Questions

(1)Write a C program to create a linked list and perform insertions and deletions of all cases.

(2) Write functions to sort and finally delete the entire list at once

Answer

```
#include <stdio.h>
#include <stdlib.h>
#include <conio.h>
#include <malloc.h>
struct node
{
        int data;
        struct node *next;
};
struct node *start = NULL;
struct node *create_ll(struct node *);
struct node *display(struct node *);
struct node *insert_beg(struct node *);
struct node *insert_end(struct node *);
struct node *insert_before(struct node *);
struct node *insert_after(struct node *);
struct node *delete_beg(struct node *);
struct node *delete_end(struct node *);
struct node *delete_node(struct node *);
struct node *delete_after(struct node *);
struct node *delete_list(struct node *);
struct node *sort_list(struct node *);
```

```
int main(int argc, char *argv[]) {
        int option;
        do
        {
                printf("\n\n *****MAIN MENU *****");
                printf("\n 1:  Create a list");
                printf("\n 2:  Display the list");
                printf("\n 3:  Add a node at the beginning");
                printf("\n 4:  Add a node at the end");
                printf("\n 5:  Add a node before a given node");
                printf("\n 6:  Add a node after a given node");
                printf("\n 7:  Delete a node from the beginning");
```

```
printf("\n 8:  Delete a node from the end");
printf("\n 9:  Delete a given node");
printf("\n 10: Delete a node after a given node");
printf("\n 11: Delete the entire list");
printf("\n 12: Sort the list");
printf("\n 13: EXIT");
printf("\n\n Enter your option : ");
scanf("%d", &option);
switch(option)
{
```

```
case 1: start = create_ll(start);
        printf("\n LINKED LIST CREATED");
        break;
case 2: start = display(start);
        break;
case 3: start = insert_beg(start);
        break;
case 4: start = insert_end(start);
        break;
case 5: start = insert_before(start);
        break;
case 6: start = insert_after(start);
        break;
case 7: start = delete_beg(start);
        break;
case 8: start = delete_end(start);
        break;
case 9: start = delete_node(start);
        break;
case 10: start = delete_after(start);
        break;
case 11: start = delete_list(start);
        printf("\n LINKED LIST DELETED");
        break;
case 12: start = sort_list(start);
        break;
}
```

```
                }
        }while(option !=13);
        getch();
        return 0;
}
struct node *create_ll(struct node *start)
{
        struct node *new_node, *ptr;
        int num;
        printf("\n Enter -1 to end");
        printf("\n Enter the data : ");
        scanf("%d", &num);
        while(num!=-1)
        {
                new_node = (struct node*)malloc(sizeof(struct node));
                new_node -> data=num;
                if(start==NULL)
                {
                        new_node -> next = NULL;
                        start = new_node;
                }
                else
                {
                        ptr=start;
```

```
                        while(ptr->next!=NULL)
                        ptr=ptr->next;
                        ptr->next = new_node;
                        new_node->next=NULL;
                }
                printf("\n Enter the data : ");
                scanf("%d", &num);
        }
        return start;
}
struct node *display(struct node *start)
{
        struct node *ptr;
        ptr = start;
        while(ptr != NULL)
        {
                printf("\t %d", ptr -> data);
                ptr = ptr -> next;
        }
        return start;
}
```

```
struct node *insert_beg(struct node *start)
{
        struct node *new_node;
        int num;
        printf("\n Enter the data : ");
        scanf("%d", &num);
        new_node = (struct node *)malloc(sizeof(struct node));
        new_node -> data = num;
        new_node -> next = start;
        start = new_node;
        return start;
}
```

```
struct node *insert_end(struct node *start)
{
        struct node *ptr, *new_node;
        int num;
        printf("\n Enter the data : ");
        scanf("%d", &num);
        new_node = (struct node *)malloc(sizeof(struct node));
        new_node -> data = num;
        new_node -> next = NULL;
        ptr = start;
        while(ptr -> next != NULL)
        ptr = ptr -> next;
        ptr -> next = new_node;
        return start;
}
```

```
struct node *insert_before(struct node *start)
{
        struct node *new_node, *ptr, *preptr;
        int num, val;
        printf("\n Enter the data : ");
        scanf("%d", &num);
        printf("\n Enter the value before which the data has to be inserted : ");
        scanf("%d", &val);
        new_node = (struct node *)malloc(sizeof(struct node));
        new_node -> data = num;
        ptr = start;
        while(ptr -> data != val)
        {
```

```
                preptr = ptr;
                ptr = ptr -> next;
        }
        preptr -> next = new_node;
        new_node -> next = ptr;
        return start;
}
```

```
struct node *insert_after(struct node *start)
{
        struct node *new_node, *ptr, *preptr;
        int num, val;
        printf("\n Enter the data : ");
        scanf("%d", &num);
        printf("\n Enter the value after which the data has to be inserted : ");
        scanf("%d", &val);
        new_node = (struct node *)malloc(sizeof(struct node));
        new_node -> data = num;
        ptr = start;
        preptr = ptr;
        while(preptr -> data != val)
        {
                preptr = ptr;
                ptr = ptr -> next;
        }
        preptr -> next=new_node;
        new_node -> next = ptr;
        return start;
}
```

```
struct node *delete_beg(struct node *start)
{
        struct node *ptr;
        ptr = start;
        start = start -> next;
        free(ptr);
        return start;
}
struct node *delete_end(struct node *start)
{
        struct node *ptr, *preptr;
        ptr = start;
        while(ptr -> next != NULL)
        {
                preptr = ptr;
                ptr = ptr -> next;
        }
        preptr -> next = NULL;
        free(ptr);
        return start;
}
```

```
struct node *delete_node(struct node *start)
{
        struct node *ptr, *preptr;
        int val;
        printf("\n Enter the value of the node which has to be deleted : ");
        scanf("%d", &val);
        ptr = start;
        if(ptr -> data == val)
        {
                start = delete_beg(start);
                return start;
        }
        else
        {
```

```
        while(ptr -> data != val)
        {
                preptr = ptr;
                ptr = ptr -> next;
        }
        preptr -> next = ptr -> next;
        free(ptr);
        return start;
}
```

```
struct node *delete_after(struct node *start)
{
        struct node *ptr, *preptr;
        int val;
        printf("\n Enter the value after which the node has to deleted : ");
        scanf("%d", &val);
        ptr = start;
        preptr = ptr;
        while(preptr -> data != val)
        {
                preptr = ptr;
                ptr = ptr -> next;
        }
        preptr -> next=ptr -> next;
        free(ptr);
        return start;
}
```

```
struct node *delete_list(struct node *start)
{
        struct node *ptr; // Lines 252-254 were modified from original code to fix
unresposiveness in output window
        if(start!=NULL){
                ptr=start;
                while(ptr != NULL)
                {
                        printf("\n %d is to be deleted next", ptr -> data);
                        start = delete_beg(ptr);
                        ptr = start;
                }
        }

        return start;
}
```

```
struct node *sort_list(struct node *start)
{
        struct node *ptr1, *ptr2;
        int temp;
        ptr1 = start;
        while(ptr1 -> next != NULL)
        {
                ptr2 = ptr1 -> next;
                while(ptr2 != NULL)
                {
                        if(ptr1 -> data > ptr2 -> data)
                        {
                                temp = ptr1 -> data;
                                ptr1 -> data = ptr2 -> data;
                                ptr2 -> data = temp;
                        }
                        ptr2 = ptr2 -> next;
                }
                ptr1 = ptr1 -> next;
```

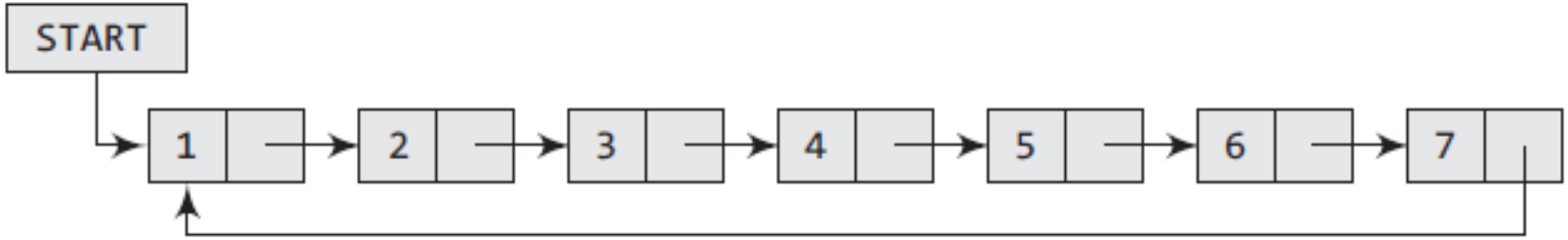
```
                    }
            return start;  // Had to be added
    }
```

Output

```
*****MAIN MENU *****
1: Create a list
2: Display the list
3: Add a node at the beginning
4: Add the node at the end
5: Add the node before a given node
6: Add the node after a given node
7: Delete a node from the beginning
8: Delete a node from the end
9: Delete a given node
10: Delete a node after a given node
11: Delete the entire list
12: Sort the list
13: Exit
Enter your option : 3
Enter your option : 73
```

CIRCULAR LINKED LISTS

- In a circular linked list, the last node contains a pointer to the first node of the list.



The only downside of a circular linked list is the complexity of iteration.
Note: there are no `NULL` values in the `NEXT` part of any of the nodes of list.

CIRCULAR LINKED LISTS cont..

- We can have a circular singly linked list as well as a circular doubly linked list.
- While traversing a circular linked list, we can begin at any node and traverse the list in any direction, forward or backward, until we reach the same node where we started.
- Circular linked lists are widely used in operating systems for task maintenance.
- Example : When we are surfing the Internet, we can use the Back button and the Forward button to move to the previous pages that we have already visited.

CIRCULAR LINKED LISTS cont..

- How is this done?
- The answer is simple: A circular linked list is used to maintain the sequence of the Web pages visited.
- Traversing this circular linked list either in forward or backward direction helps to revisit the pages again using Back and Forward buttons.
- Actually, this is done using either the circular stack or the circular queue

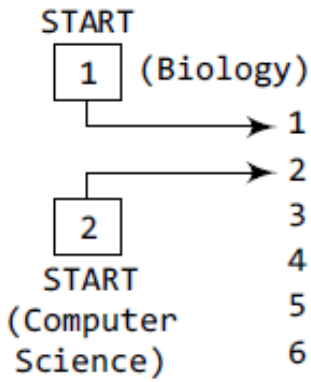
Memory representation of a circular linked list

START

1

	DATA	NEXT
1	H	4
2		
3		
4	E	7
5		
6		
7	L	8
8	L	10
9		
10	O	1

Memory representation of two circular linked lists stored in the memory



	Roll No	NEXT
1	S01	3
2	S02	5
3	S03	8
4		
5	S04	7
6		
7	S05	10
8	S06	11
9		
10	S07	12
11	S08	13
12	S09	1
13	S10	15
14		
15	S11	2

Inserting a New Node in a Circular Linked List

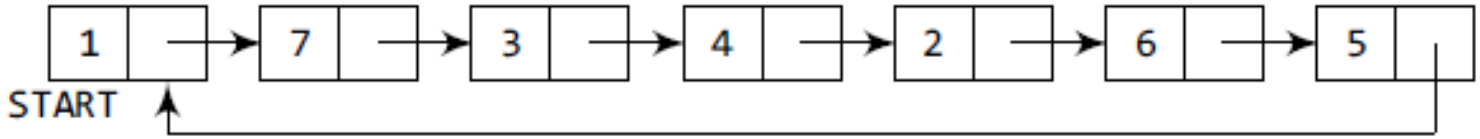
- We will take two cases and then see how insertion is done in each case.

Case 1: The new node is inserted at the beginning of the circular linked list.

Case 2: The new node is inserted at the end of the circular linked list.

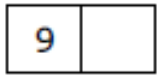
Inserting a New Node in a Circular Linked List

- Case 1: The new node is inserted at the beginning of the circular linked list.

- 

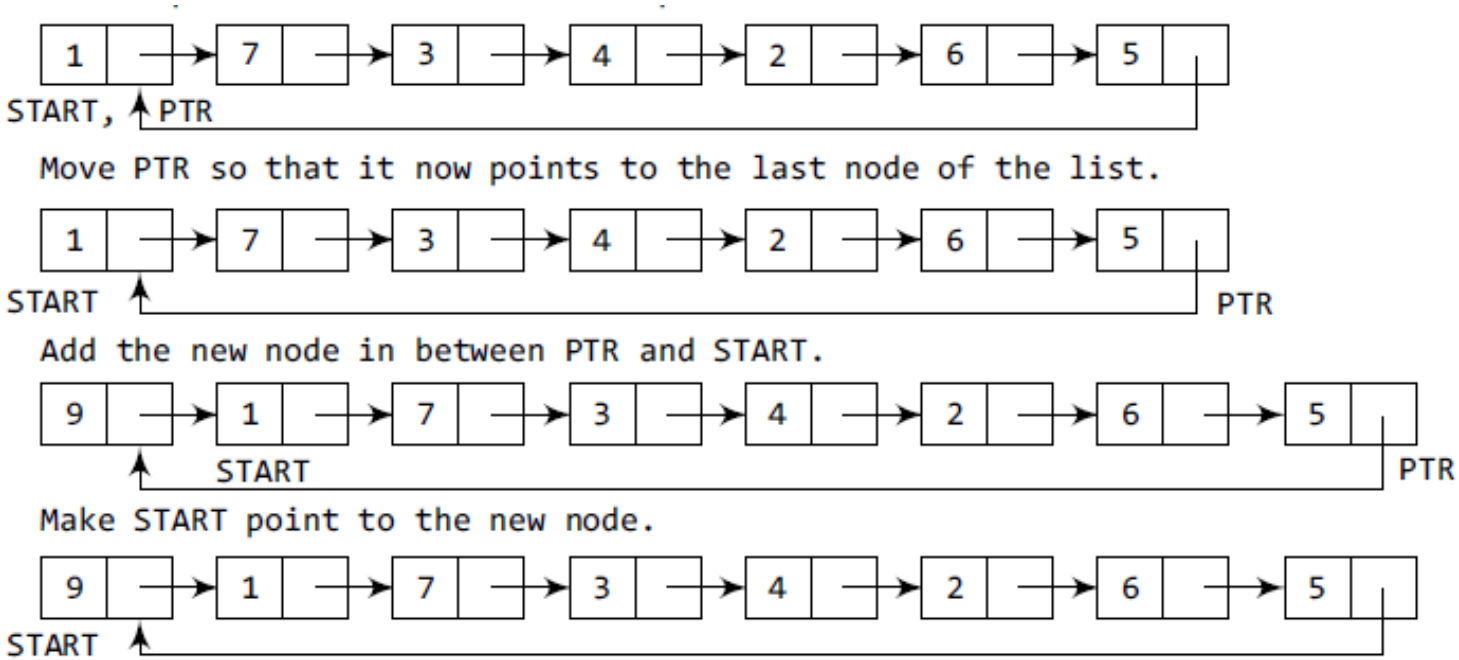
- Then the following changes will be done in the linked list.

Allocate memory for the new node and initialize its DATA part to 9.



Take a pointer variable PTR that points to the START node of the list.

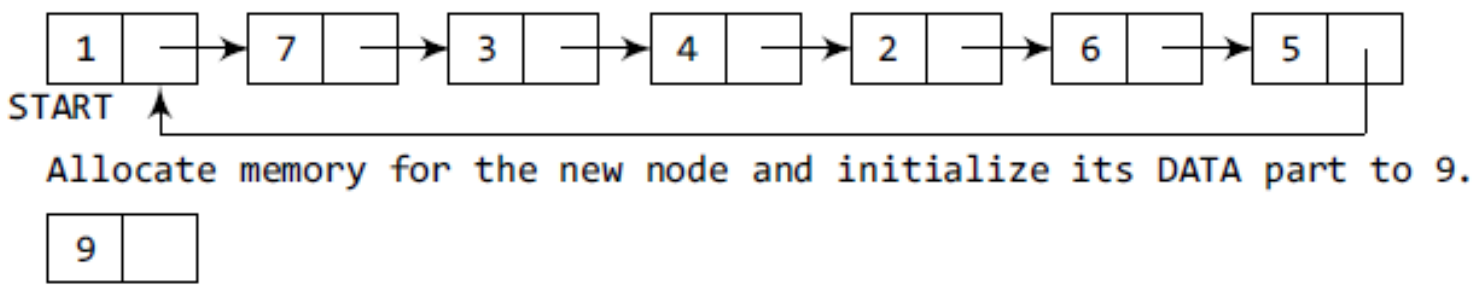
Case 1: The new node is inserted at the beginning of the circular linked list



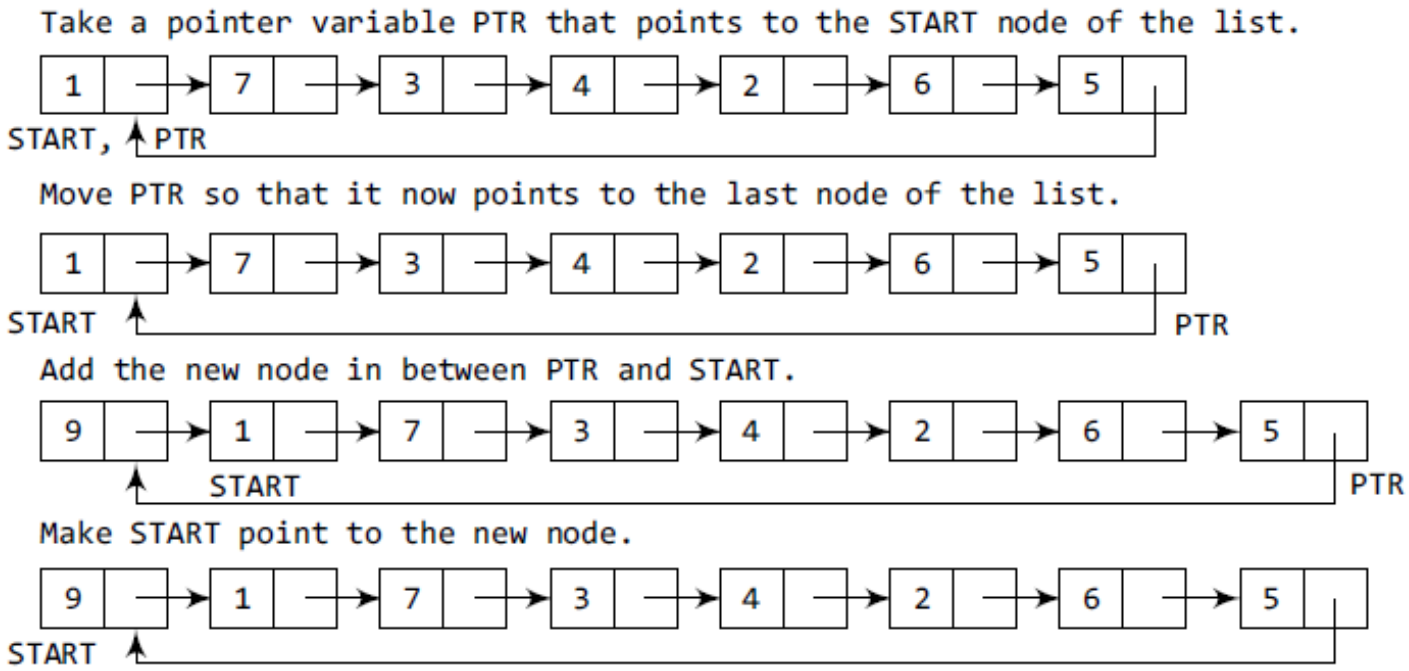
Case 1: The new node is inserted at the beginning of the circular linked list

```
Step 1: IF AVAIL = NULL
            Write OVERFLOW
            Go to Step 11
       [END OF IF]
Step 2: SET NEW_NODE = AVAIL
Step 3: SET AVAIL = AVAIL -> NEXT
Step 4: SET NEW_NODE -> DATA = VAL
Step 5: SET PTR = START
Step 6: Repeat Step 7 while PTR -> NEXT != START
Step 7:     PTR = PTR -> NEXT
        [END OF LOOP]
Step 8: SET NEW_NODE -> NEXT = START
Step 9: SET PTR -> NEXT = NEW_NODE
Step 10: SET START = NEW_NODE
Step 11: EXIT
```

Inserting a Node at the End of a Circular Linked List

- Case 2: The new node is inserted at the end of the circular linked list.
- Suppose we want to add a new node with data 9 as the last node of the list.
- 

Inserting a Node at the End of a Circular Linked List cont..



Inserting a Node at the End of a Circular Linked List cont..

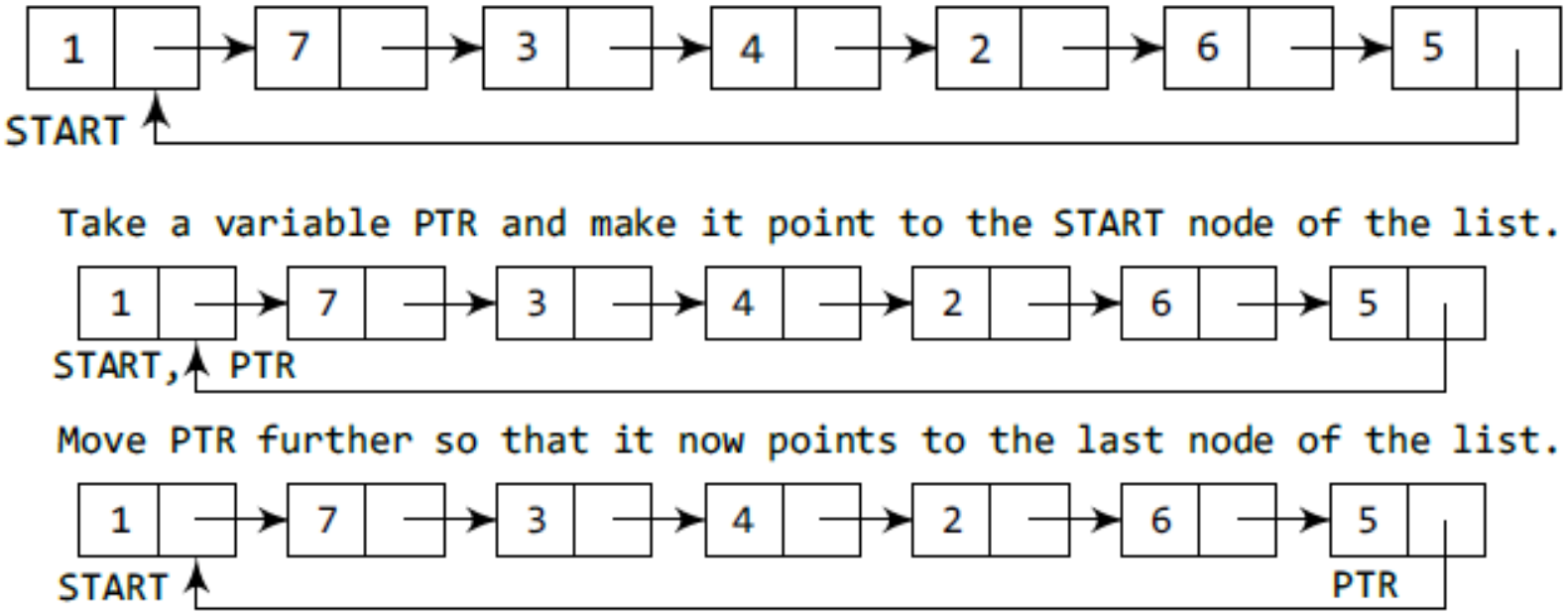
```
Step 1: IF AVAIL = NULL
            Write OVERFLOW
            Go to Step 10
       [END OF IF]
Step 2: SET NEW_NODE = AVAIL
Step 3: SET AVAIL = AVAIL -> NEXT
Step 4: SET NEW_NODE -> DATA = VAL
Step 5: SET NEW_NODE -> NEXT = START
Step 6: SET PTR = START
Step 7: Repeat Step 8 while PTR -> NEXT != START
Step 8:     SET PTR = PTR -> NEXT
       [END OF LOOP]
Step 9: SET PTR -> NEXT = NEW_NODE
Step 10: EXIT
```

Deleting a Node from a Circular Linked List

- We will take two cases and then see how deletion is done in each case.
- Rest of the cases of deletion are same as that given for singly linked lists.
- Case 1: The first node is deleted.
- Case 2: The last node is deleted

Deleting a Node from a Circular Linked List

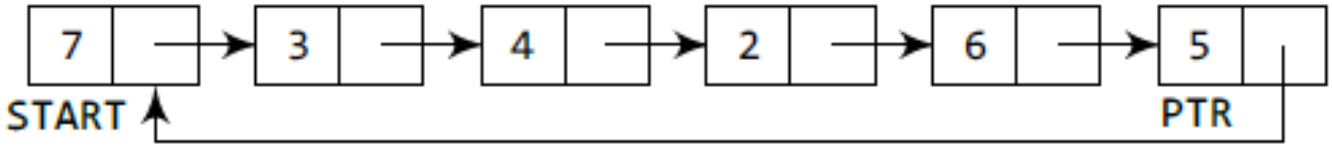
- Case 1: The first node is deleted.



Deleting a Node from a Circular Linked List cont..

- Case 1: The first node is deleted cont.....

The NEXT part of PTR is made to point to the second node of the list and the memory of the first node is freed. The second node becomes the first node of the list.

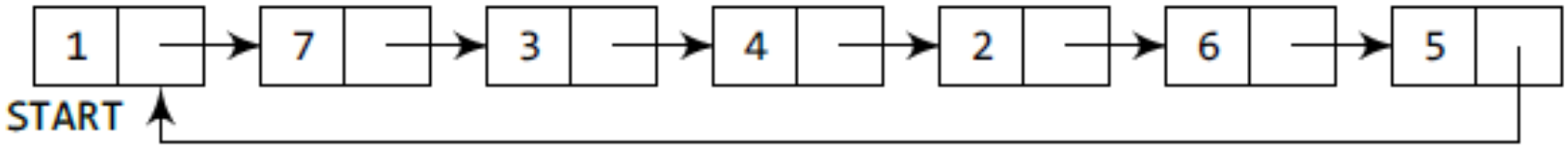


Case 1: The first node is deleted cont.....

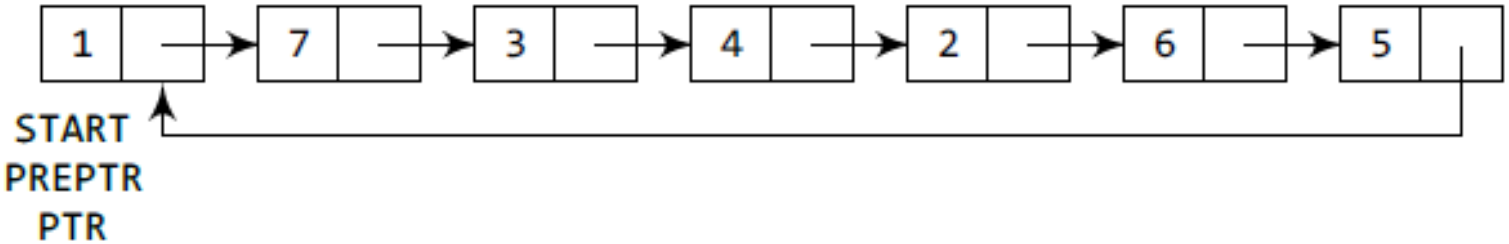
```
Step 1: IF START = NULL
              Write UNDERFLOW
              Go to Step 8
         [END OF IF]
Step 2: SET PTR = START
Step 3: Repeat Step 4 while PTR -> NEXT != START
Step 4:       SET PTR = PTR -> NEXT
        [END OF LOOP]
Step 5: SET PTR -> NEXT = START -> NEXT
Step 6: FREE START
Step 7: SET START = PTR -> NEXT
Step 8: EXIT
```

Deleting the Last Node from a Circular Linked List

- Case 2: The last node is deleted

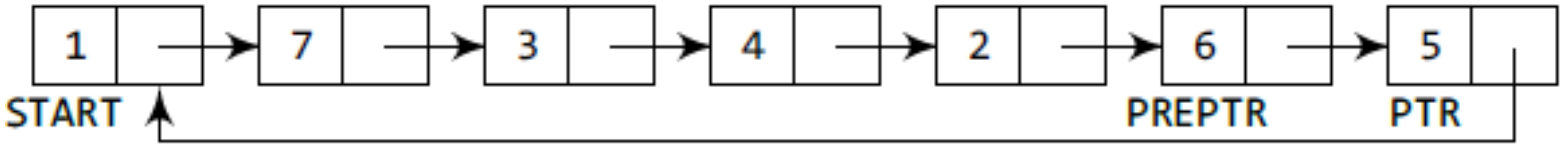


Take two pointers PREPTR and PTR which will initially point to START.

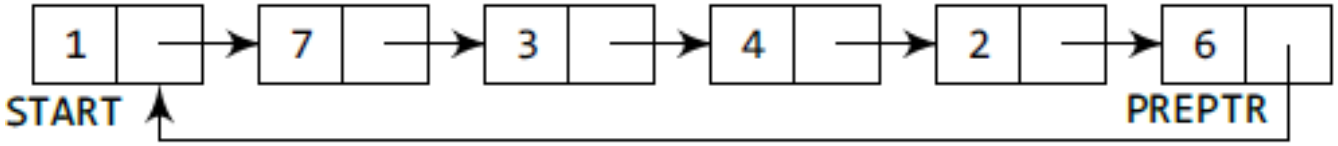


Deleting the Last Node from a Circular Linked List

Move PTR so that it points to the last node of the list. PREPTR will always point to the node preceding PTR.



Make the PREPTR's next part store START node's address and free the space allocated for PTR. Now PREPTR is the last node of the list.



Algorithm to delete the last node

```
Step 1: IF START = NULL
                Write UNDERFLOW
                Go to Step 8
          [END OF IF]
Step 2: SET PTR = START
Step 3: Repeat Steps 4 and 5 while PTR -> NEXT != START
Step 4:         SET PREPTR = PTR
Step 5:         SET PTR = PTR -> NEXT
          [END OF LOOP]
Step 6: SET PREPTR -> NEXT = START
Step 7: FREE PTR
Step 8: EXIT
```

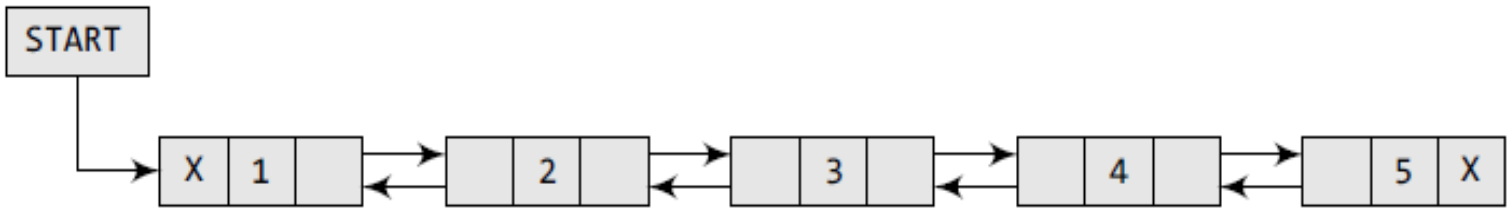
Home work

- Write a program to create a circular linked list. Perform insertion and deletion at the beginning and end of the list.

DOUBLY LINKED LISTS

- A doubly linked list or a two-way linked list is a more complex type of linked list which contains a pointer to the next as well as the previous node in the sequence.
- Doubly linked list cell consists of three parts:
- data,
- a pointer to the next node,
- and a pointer to the previous node

DOUBLY LINKED LISTS cont….



In C, the structure of a doubly linked list can be given as,

```
struct node
{
        struct node *prev;
        int data;
        struct node *next;
};
```

Memory representation of a doubly linked list

START

1

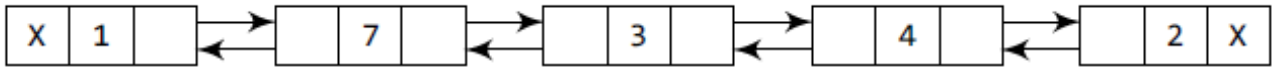
	DATA	PREV	NEXT
1	H	−1	3
2			
3	E	1	6
4			
5			
6	L	3	7
7	L	6	9
8			
9	O	7	−1

Inserting a New Node in a Doubly Linked List

- We will take four cases and then see how insertion is done in each case.
- Case 1: The new node is inserted at the beginning.
- Case 2: The new node is inserted at the end.
- Case 3: The new node is inserted after a given node.
- Case 4: The new node is inserted before a given node.

Case 1: Inserting a Node at the Beginning of a Doubly Linked List cont….

- Suppose we want to add a new node with data 9 as the first node of the list.
- Then the following changes will be done in the linked list.

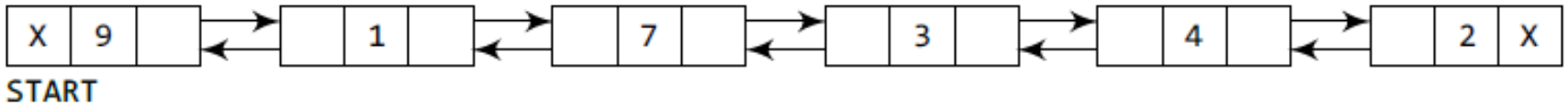


START

Allocate memory for the new node and initialize its DATA part to 9 and PREV field to NULL.

Case 1: Inserting a Node at the Beginning of a Doubly Linked List cont.

Add the new node before the START node. Now the new node becomes the first node of the list.



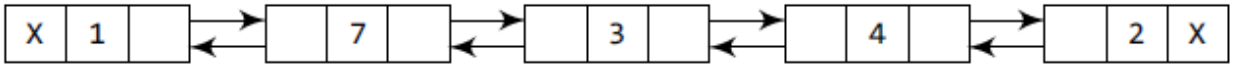
Algorithm to insert a new node at the beginning

```
Step 1: IF AVAIL = NULL
                Write OVERFLOW
                Go to Step 9
        [END OF IF]
Step 2: SET NEW_NODE = AVAIL
Step 3: SET AVAIL = AVAIL -> NEXT
Step 4: SET NEW_NODE -> DATA = VAL
Step 5: SET NEW_NODE -> PREV = NULL
Step 6: SET NEW_NODE -> NEXT = START
Step 7: SET START -> PREV = NEW_NODE
Step 8: SET START = NEW_NODE
Step 9: EXIT
```

Case 2:*Inserting a Node at the End of a Doubly Linked List*

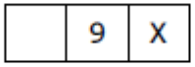
- Suppose we want to add a new node with data 9 as the last node of the list.
- Then the following changes will be done in the linked list

Case 2:***Inserting a Node at the End of a Doubly Linked List*** cont..

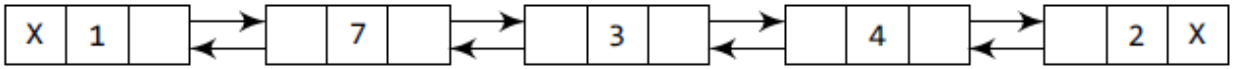


START

Allocate memory for the new node and initialize its DATA part to 9 and its NEXT field to NULL.

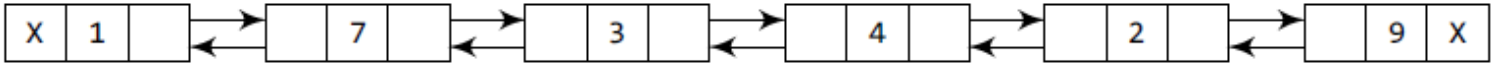


Take a pointer variable PTR and make it point to the first node of the list.



START,PTR

Move PTR so that it points to the last node of the list. Add the new node after the node pointed by PTR.



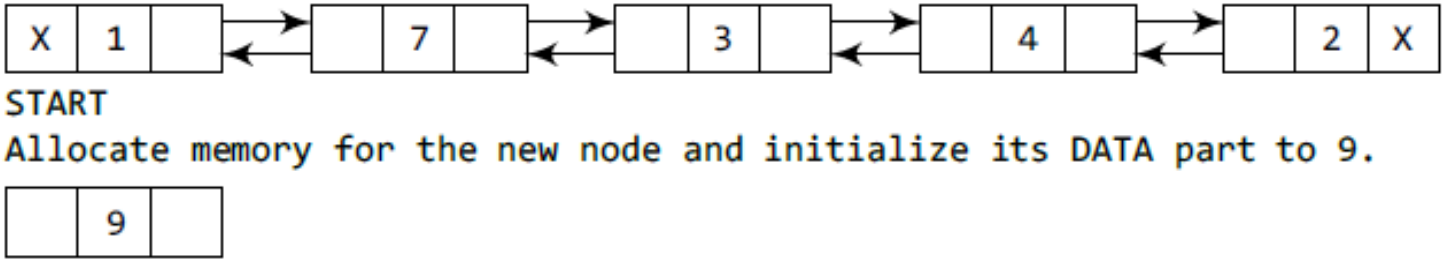
START PTR

Algorithm to insert a new node at the end

```
Step 1: IF AVAIL = NULL
            Write OVERFLOW
            Go to Step 11
       [END OF IF]
Step 2: SET NEW_NODE = AVAIL
Step 3: SET AVAIL = AVAIL -> NEXT
Step 4: SET NEW_NODE -> DATA = VAL
Step 5: SET NEW_NODE -> NEXT = NULL
Step 6: SET PTR = START
Step 7: Repeat Step 8 while PTR -> NEXT != NULL
Step 8:     SET PTR = PTR -> NEXT
       [END OF LOOP]
Step 9: SET PTR -> NEXT = NEW_NODE
Step 10: SET NEW_NODE -> PREV = PTR
Step 11: EXIT
```

Inserting a Node After a Given Node in a Doubly Linked List

- Suppose we want to add a new node with value 9 after the node containing 3.



Inserting a Node After a Given Node in a Doubly Linked List

Take a pointer variable PTR and make it point to the first node of the list.

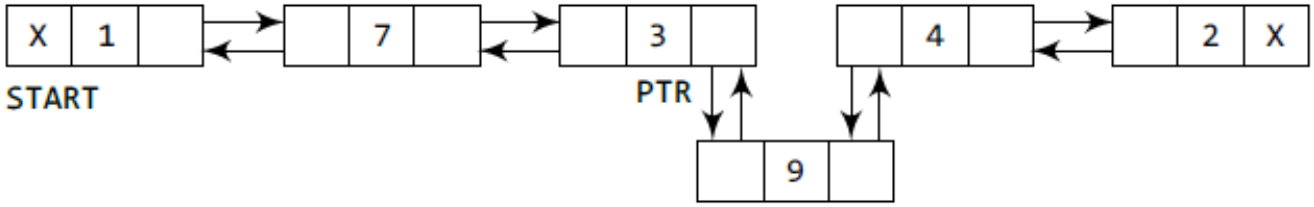
START,PTR

Move PTR further until the data part of PTR = value after which the node has to be inserted.

START

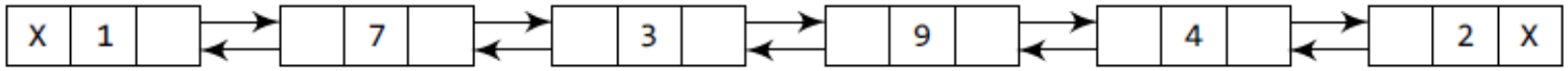
PTR

Insert the new node between PTR and the node succeeding it.



START

PTR



START

Algorithm to insert a new node after a given node

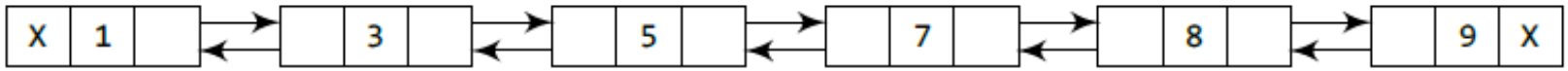
```
Step 1: IF AVAIL = NULL
            Write OVERFLOW
            Go to Step 12
       [END OF IF]
Step 2: SET NEW_NODE = AVAIL
Step 3: SET AVAIL = AVAIL -> NEXT
Step 4: SET NEW_NODE -> DATA = VAL
Step 5: SET PTR = START
Step 6: Repeat Step 7 while PTR -> DATA != NUM
Step 7:     SET PTR = PTR -> NEXT
       [END OF LOOP]
Step 8: SET NEW_NODE -> NEXT = PTR -> NEXT
Step 9: SET NEW_NODE -> PREV = PTR
Step 10: SET PTR -> NEXT = NEW_NODE
Step 11: SET PTR -> NEXT -> PREV = NEW_NODE
Step 12: EXIT
```

Deleting a Node from a Doubly Linked List

- We will take four cases and then see how deletion is done in each case.

Case 1: The first node is deleted.
Case 2: The last node is deleted.
Case 3: The node after a given node is deleted.
Case 4: The node before a given node is deleted.

Deleting the First Node from a Doubly Linked List



START

Free the memory occupied by the first node of the list and make the second node of the list as the START node.

START

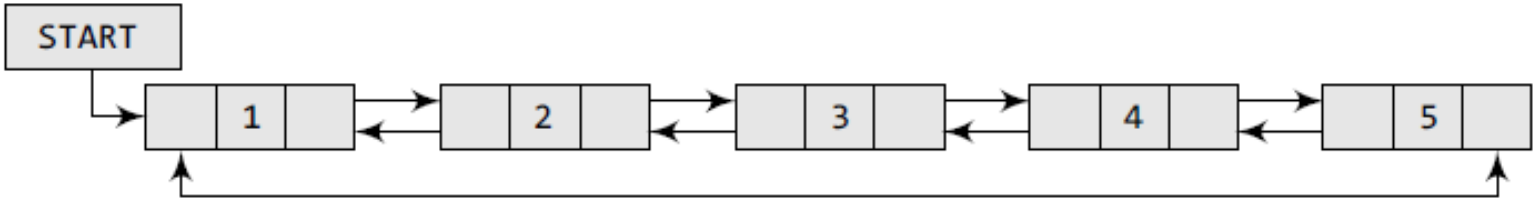
Algorithm to delete the first node

```
Step 1: IF START = NULL
            Write UNDERFLOW
            Go to Step 6
        [END OF IF]
Step 2: SET PTR = START
Step 3: SET START = START -> NEXT
Step 4: SET START -> PREV = NULL
Step 5: FREE PTR
Step 6: EXIT
```

HOMEWORK

- Write a program to create a doubly linked list and perform insertions and deletions in all cases.

CIRCULAR DOUBLY LINKED LISTs



Memory representation of a circular doubly linked list

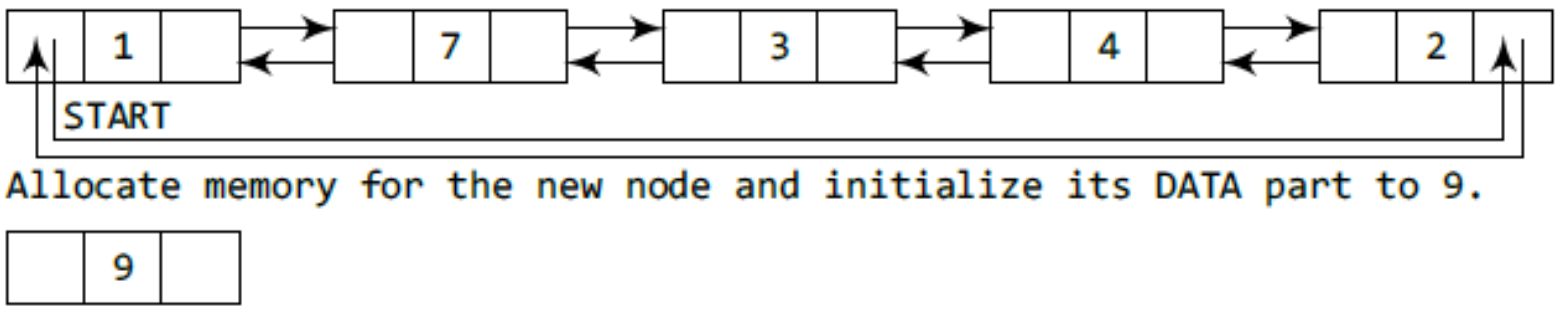
START

1

	DATA	PREV	Next
1	H	9	3
2			
3	E	1	6
4			
5			
6	L	3	7
7	L	6	9
8			
9	O	7	1

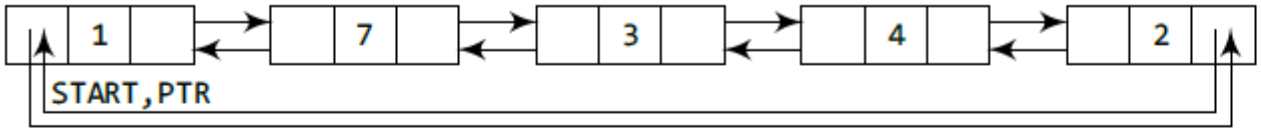
Inserting a New Node in a Circular Doubly Linked List

- ***Inserting a Node at the Beginning of a Circular Doubly Linked List***
- Suppose we want to add a new node with

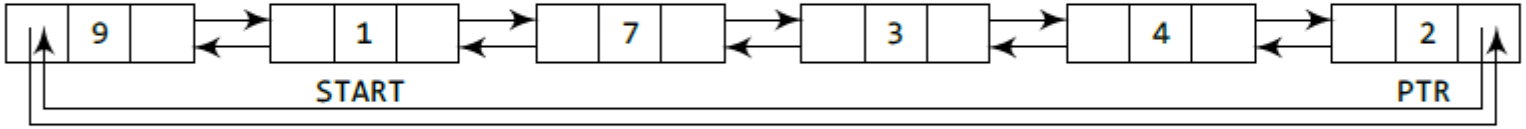


Inserting a New Node in a Circular Doubly Linked List

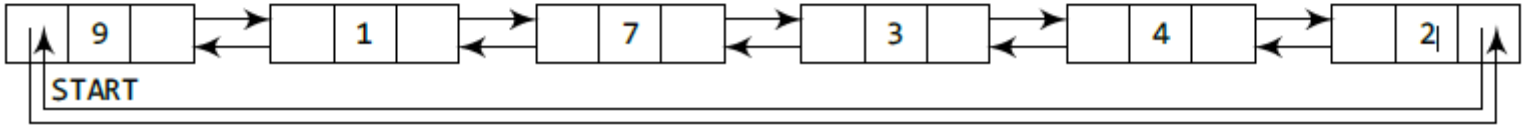
Take a pointer variable PTR that points to the first node of the list.



Move PTR so that it now points to the last node of the list. Insert the new node in between PTR and the START node.



START will now point to the new node.



Algorithm to insert a new node at the beginning

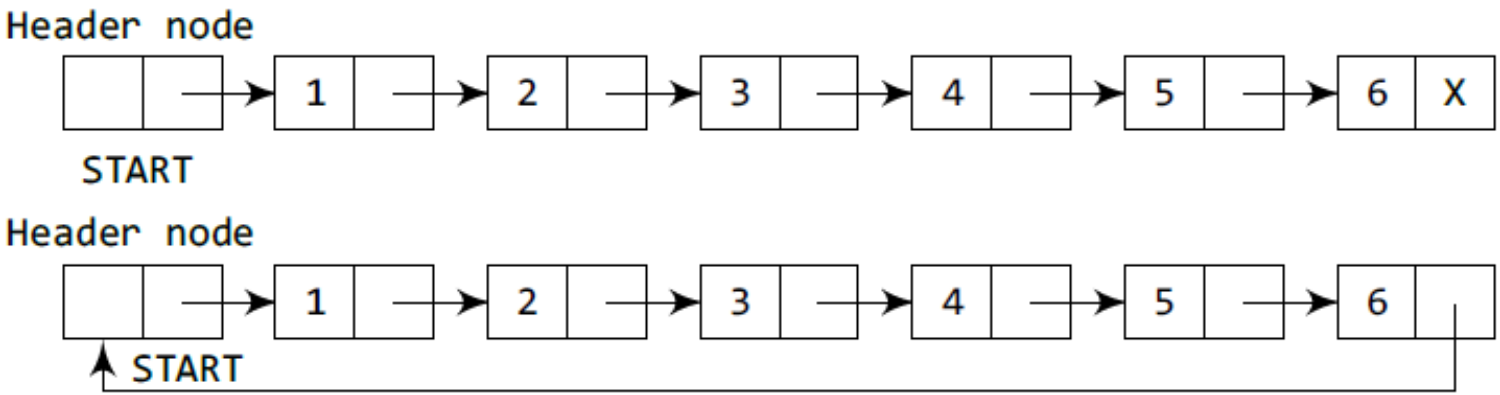
```
Step 1: IF AVAIL = NULL
            Write OVERFLOW
            Go to Step 13
       [END OF IF]
Step 2: SET NEW_NODE = AVAIL
Step 3: SET AVAIL = AVAIL -> NEXT
Step 4: SET NEW_NODE -> DATA = VAL
Step 5: SET PTR = START
Step 6: Repeat Step 7 while PTR -> NEXT != START
Step 7:     SET PTR = PTR -> NEXT
       [END OF LOOP]
Step 8: SET PTR -> NEXT = NEW_NODE
Step 9: SET NEW_NODE -> PREV = PTR
Step 10: SET NEW_NODE -> NEXT = START
Step 11: SET START -> PREV = NEW_NODE
Step 12: SET START = NEW_NODE
Step 13: EXIT
```

Write a program to create a circular doubly linked list and perform insertions and deletions
at the beginning and end of the list.

Header Linked Lists

- A header linked list is a special type of linked list which contains a header node at the beginning of the list.
- So, in a header linked list, START will not point to the first node of the list but START will contain the address of the header node.
- The following are the two variants of a header linked list:
- *Grounded header linked list* which stores NULL in the next field of the last node.
- *Circular header linked list* which stores the address of the header node in the next field of the last node

Header Linked Lists



Memory representation of a header linked list

START

5

	DATA	NEXT
1	H	3
2		
3	E	6
4		
5	1234	1
6	L	7
7	L	9
8		
9	0	-1

Memory representation of a circular header linked list

START

5

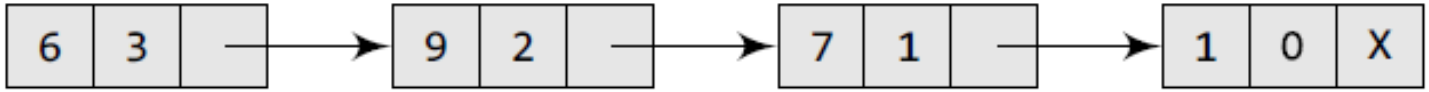
	DATA	NEXT
1	H	3
2		
3	E	6
4		
5	1234	1
6	L	7
7	L	9
8		
9	O	5

HOMEWORK

- Write a program to implement a header linked list.

APPLICATIONS OF LINKED LISTS

- **Polynomial Representation**
- Consider a polynomial 6x3 + 9x2 + 7x + 1.
- Every individual term in a polynomial consists of two parts, a coefficient and a power.



HOMEWORK

- Write a program to store a polynomial using linked list. Also, perform addition and subtraction on two polynomials